Orientación a objetos en PHP

Definición de clases en PHP

En PHP, una clase se define con la palabra clave `class`, seguida del nombre de la clase y un par de llaves ({}). Todas sus propiedades y métodos van entre las llaves:

```php
<?php

class Persona {
    // Propiedades (atributos)
    public string $nombre;
    public int $edad;

   // Constructor public
   function __construct(string $nombre, int $edad) {
       $this->nombre = $nombre;
       $this->edad = $edad;
   }

   // Métodos public
   function presentarse(): string {
       return "Hola, soy {$this->nombre} y tengo {$this->edad} años.";
   }
}
```

La palabra clave `$this` se refiere al objeto actual y solo está disponible dentro de los métodos.

Un constructor permite inicializar las propiedades de un objeto al crearlo. Si creamos una función `__construct()`, PHP la llamará automáticamente al crear un objeto desde una clase.

Existe un método `__destruct()` que se ejecuta al destruir el objeto, pero en PHP se usa poco porque el propio motor gestiona la memoria.

Recuerda que la función `construct` y `destruct` comienzan con dos guiones bajos (__). En **PHP NO existe sobrecarga de constructores** como en Java o C#. Es decir, **solo puede haber un `__construct()` por clase**.

Los atributos y métodos pueden ser declarados como `static`.

Para acceder a una propiedad o un método usaremos el operador `->`. Para acceder a un atributo o método estático no se usa `->` sino `::`. Cuando estamos **dentro de la clase**, se

suele usar `self::`, hace referencia a la clase donde estás escribiendo el código.

```
class Padre {
    public static function quienSoy() {
        return "Soy Padre";
    }

    public static function test() {
        return self::quienSoy();   // siempre "Soy Padre"
    }

    public static function testLateStatic() {
        return static::quienSoy(); // respeta la herencia
    }
}

class Hijo extends Padre {
    public static function quienSoy() {
        return "Soy Hijo";
    }
}

echo Hijo::test();            // Soy Padre
echo Hijo::testLateStatic();  // Soy Hijo
```

Creación de objetos

Para instanciar objetos se usa `new`:

```
$alumno = new Persona("Inma", 30);
echo $alumno->presentarse();
```

Podemos utilizar la palabra clave `instanceof` para comprobar si un objeto pertenece a una clase específica:

```
<?php
$alumno = new Persona("Inma", 30);
var_dump($alumno instanceof Persona);
```

Modificadores de acceso

Las propiedades y los métodos pueden tener modificadores de acceso que controlan dónde se puede acceder a ellos. Existen tres modificadores de acceso:

- Público: se puede acceder a la propiedad o método desde cualquier lugar. Esta es la configuración predeterminada.
- Protegido: se puede acceder a la propiedad o método dentro de la clase y por las clases derivadas de ella.
- Privado: se puede acceder a la propiedad o método SOLO dentro de la clase.

PHP soporta **public**, **protected** y **private**:

```php
class CuentaBancaria {
    private float $saldo = 0;
    public function depositar(float $cantidad): void {
        $this->saldo += $cantidad;
    }
    public function consultarSaldo(): float {
        return $this->saldo;
    }
}
```

Tipos de los atributos

Como ya sabemos PHP es un lenguaje no tipado, sin embargo a partir de PHP 7.4, se introdujeron los atributos y variables tipados, pero no son obligatorios. Omitir el tipo da más flexibilidad, pero reduce la seguridad del código:

- No hay comprobación automática de tipo.
- Es más fácil introducir errores silenciosos.

Por eso, en código moderno (PHP 8+), se recomienda tipar siempre que sea posible. Vamos a ver una tabla con los tipos básicos que podemos usar en PHP.ables**:

Tipo	Descripción	Ejemplo
`int`	Entero	`public int $edad;`
`float`	Decimal	`function setPrecio(float $p)`
`string`	Cadena	`private string $nombre;`
`bool`	Booleano	`public bool $activo;`
`array`	Array	`public array $valores;`
`object`	Cualquier objeto	`function procesar(object $o)`

En PHP existe un modificador de tipo, el símbolo `?`. El prefijo `?` delante de un tipo indica que el valor puede ser de ese tipo o `null`. Es decir, `?int` significa que la variable puede ser un entero o `null`.

Herencia

Herencia en programación orientada a objetos (POO) se utiliza cuando una clase deriva de otra. La clase hija heredará todas las propiedades y métodos públicos y protegidos de la clase padre. Además, puede tener sus propias propiedades y métodos. Una clase heredada se define mediante la palabra clave `extends`.

```php
class Estudiante extends Persona {
    private string $curso;
    public function __construct(string $nombre, int $edad, string $curso) {
        parent::__construct($nombre, $edad); // llamada al constructor padre
        $this->curso = $curso;
    }

    public function presentarse(): string {
        return parent::presentarse() . " Estudio en el curso {$this->curso}.";
    }
}
```

Interfaces y clases abstractas

En PHP se definen con `interface` y `abstract`:

```php
interface Enviable {
    public function enviar(): void;
}

abstract class Mensaje {
    abstract public function getContenido(): string;
}

class Email extends Mensaje implements Enviable {
    private string $contenido;
    public function __construct(string $contenido) {
        $this->contenido = $contenido;
    }
    public function getContenido(): string {
        return $this->contenido;
    }
    public function enviar(): void {
        echo "Enviando email: " . $this->getContenido();
    }
}
```